Copying projects and making projects from a ROM or a patch

Hawk's Hangar (Step 33). The start screen and the File menu offer three ways to get a project without starting from scratch. All three ask for a name and a location in the same dialog as "New project" and then open the new project.

Command	Start screen	File menu
Copy project…	asks for the project folder first	copies the open project (unsaved edits are saved or discarded first)
Project from ROM…	asks for a hack ROM (`.z64` , `.v64` , `.n64`)	same
Project from patch…	asks for a BPS patch	same

The File menu starts with **Back to the start screen** (it replaces "Close project", which did the same: unsaved edits are offered for saving, then the start window opens again).

1. Copy project (`HackProject.CopyTo` , `EditorSession.CopyTo`)

- Copied: `project.json` (with the new name; a deep copy, so the projects share nothing), the whole `files/` folder (replacement files, imported songs in `files/music/` , anything else the user keeps there), and `project.user.json` with the clean ROM path.
- Not copied: `backups/` (restore points contain ROMs), the built hack ROM and any other ROM in the project folder. The output ROM path is reset to the default (inside the new folder), so the copy never overwrites the original's ROM.
- `.gitignore` : the Git box of the dialog decides. Checked: the original's `.gitignore` , or the default one if it has none. The box starts checked when the original has one.
- The target must be a new or empty folder and must not lie inside the original project.
- The default location is the folder that contains the original; the default name is "Name (copy)".

2. Project from ROM (`ProjectImporter`)

`ProjectImporter.Analyze(hack, clean, layout, fallbackName, patchMetadata)` compares the hack ROM with the clean ROM; `ProjectImporter.CreateProject` writes the project; `EditorSession.CreateFromImport` does both steps, builds the new project once and compares the build with the imported ROM (`ProjectImporter.Compare`). The result is a report shown in a text window (copyable).

The ROM must be Pilotwings 64 (USA): same game code and version byte in the header as the clean ROM. Byte order (`.v64` , `.n64`) is converted when the ROM is loaded.

2.1 What is taken over from every ROM

What	How
Game files	every file whose content differs from the clean ROM (same table index and type) becomes a replacement file in `files/` with the standard name. The text file is one of them, so all texts stay exactly as in the ROM. Exception: with the game patch "Bonus games and Birdman" a text file that differs from the original only by the nine level texts the build adds by itself (`ResultTables.AddLevelTexts`) is not taken over.
Songs	the music file is found through the code references of the audio layout (it may have been moved); every sequence that differs becomes an imported song (`files/music/song_NN.seq`), byte for byte.
Code fixes and game patches	each piece of each fix/patch is checked. A fix counts if all pieces are replaced. Pieces at places the game module also changes (memory manager, code cave) are judged by the other pieces. Calls into the game module (`jal` into 0x803805E0–0x803905E0) may go to other addresses (another module version). Partly present: warning, not taken over (the user can apply it in the project settings).
Build options	"Always move the audio data" if the music file was moved although the files would fit in front of it; "Allow expansion" if the ROM is larger than 16 MiB.

Not possible with a project (reported as warnings): additional game files, missing files or files of another type at a table index, other changes of the game code, changed instruments or samples, a different number of songs, a song larger than the game's room.

2.2 What is taken over from a ROM built by Hawk's Hangar

A ROM built by the editor has the game module (`'HHGM'` at ROM size − 0x10000, `GameModule.ReadHeader`):

What	Where
Name and version	information line (+0x30): `project "Name" 1.0, built yyyy-MM-dd HH:mm`
Author	only from a patch: its metadata `Name Version by Author - made with Hawk's Hangar for Pilotwings 64 (USA)`
Features	flags (+0x0C: 1 startup info, 2 game messages, 4 debug output, 8 on screen, 16 live values) and the live values (+0x90)
Conditions (also time points after a crash), unlimited fuel, hard end time scoring and jingle, the Gyrocopter's touchdown curve, pad radius and speed limits (stage 4)	the mission table: module version ⩾ 6 in the data area image (+0xE8 offset, +0xEC size, magic `'HHMD'`, entries of 0x100 bytes); older modules (version 4/5) in the module itself (+0x28 RAM address, +0x2C count, entries of 0x14 bytes, no jingle). Test builds of version 5 that already had the data area are recognised by its magic number.
Result rows	the table in the code (ROM 0xD73B0), compared per mission with the clean ROM
Added and removed missions (Step 41)	the mission list of the lower data area (`DataExtension`): files behind the game's become replacement files, the game's missions missing from the list `RemovedMissions`; rows of new places and photo missions from its tables
Hidden "View Demo" buttons	the three patched places of `DemoButton`, compared with the patches for each of the 7 combinations of HG/RB/GC

Reconstruction rules:

- An entry with point conditions becomes `MissionSettings.Conditions` (0xFF = all). An entry only for fuel or the hard end time leaves `Conditions` null: the condition byte in the (replaced) mission file keeps counting, as before.
- `HardEndZero` from flag 32. The jingle is stored only if it differs from the default of the scoring (`HardEndJingles.Effective`).
- `GenerateTexts` stays false and the footnote unset, so the editor does not rewrite any text until the user changes a mission's points (then the footnote is taken from the current mission text, as for every mission without settings).
- Text points (`TextPoints`), the description and the per-song instrument settings cannot be recovered; the changed songs are kept as whole imported songs instead.

2.3 The comparison after the import

The new project is built at once (this also writes its hack ROM) and compared byte for byte with the imported ROM. Ignored: the information line of the module (it holds the build time).

Places closer than 16 bytes are merged; the report lists per region the number of bytes, places and the first offset. Regions: ROM header, boot code (0x40–0x1000), game code (0x1000–file table), game files, music and sounds, game module and data area (also where the imported ROM had its module, for 8 MiB ROMs of older versions), other data.

Expected differences are notes, not warnings:

- every project builds a 16 MiB ROM (older hacks and foreign ROMs have 8 MiB);
- if the imported ROM has no module or another module version: the module's changes of the game code, the module itself and the module calls of game patches;
- the boot checksum, when the code of the checksummed first MiB differs anyway.

A hack built by the current editor rebuilds identically ("The project builds exactly the imported ROM"), checked by `ProjectImporterTests` with debug output, live values, combined conditions, hard end times with 0 points and with points, no jingle, unlimited fuel, swapped result rows, a hidden demo button and an imported song. A hack built with the Step 31 editor (module version 4) was checked by hand: conditions, fuel, result rows and the demo button were recovered, and the rebuilt ROM booted in ares with the imported name in the debug output.

3. Project from patch

`ProjectImporter.ApplyPatch` checks that the patch was made for the clean ROM (source size and CRC-32 in the patch), applies it (the BPS reader also checks the target CRC), and the result is imported as in section 2. The patch file name is the fallback name; the patch metadata gives the author for patches made by Hawk's Hangar.